

在 `AndroidPdfViewer` 中直接添加注释和高亮功能需要自定义开发，因为该库并未原生支持这些功能。不过，可以通过在 `PDFView` 的 `onDraw()` 方法中叠加自定义绘制层来实现注释和高亮功能。以下是详细的实现步骤：

实现步骤

1. 实现注释和高亮的基础功能

`PDFView` 提供了 `onDraw()` 方法，你可以通过它在每一页上绘制额外的内容，例如高亮区域或注释。

代码实现：高亮绘制

1. 自定义高亮数据结构 创建一个类存储每个页面的高亮区域：

```
1  java
2
3
4  复制代码
5  public class Highlight {
6      int pageIndex;
7      RectF highlightArea; // 以页面坐标表示高亮区域
8      int color;           // 高亮颜色
9
10     public Highlight(int pageIndex, RectF highlightArea, int color) {
11         this.pageIndex = pageIndex;
12         this.highlightArea = highlightArea;
13         this.color = color;
14     }
15 }
```

2. 在 `PDFView` 中叠加高亮

```
1  java
2
3
4  复制代码
5  List<Highlight> highlights = new ArrayList<>();
6
7  @Override
8  public void onDraw(Canvas canvas, float pageWidth, float pageHeight, int
displayedPage) {
9      super.onDraw(canvas, pageWidth, pageHeight, displayedPage);
10
11     for (Highlight highlight : highlights) {
12         if (highlight.pageIndex == displayedPage) {
13             // 转换页面坐标为实际屏幕坐标
14             RectF screenRect = new RectF(
15                 highlight.highlightArea.left * pageWidth,
16                 highlight.highlightArea.top * pageHeight,
17                 highlight.highlightArea.right * pageWidth,
```

```

18         highlight.highlightArea.bottom * pageHeight
19     );
20     Paint paint = new Paint();
21     paint.setColor(highlight.color);
22     paint.setStyle(Paint.Style.FILL);
23     paint.setAlpha(100); // 透明度
24     canvas.drawRect(screenRect, paint);
25 }
26 }
27 }

```

3. 添加高亮 提供一个方法供外部调用，用于向页面添加高亮：

```

1 java
2
3
4 复制代码
5 public void addHighlight(int pageIndex, RectF area, int color) {
6     highlights.add(new Highlight(pageIndex, area, color));
7     invalidate(); // 重新绘制
8 }

```

2. 实现注释功能

步骤

1. 注释数据结构 创建一个类用于存储注释内容：

```

1 java
2
3
4 复制代码
5 public class Annotation {
6     int pageIndex;
7     RectF position; // 注释显示位置
8     String content; // 注释内容
9
10    public Annotation(int pageIndex, RectF position, String content) {
11        this.pageIndex = pageIndex;
12        this.position = position;
13        this.content = content;
14    }
15 }

```

2. 在 PDFView 中叠加注释

```

1 java
2
3

```

```

4  复制代码
5  List<Annotation> annotations = new ArrayList<>();
6
7  @Override
8  public void onDraw(Canvas canvas, float pageWidth, float pageHeight, int
displayedPage) {
9      super.onDraw(canvas, pageWidth, pageHeight, displayedPage);
10
11     for (Annotation annotation : annotations) {
12         if (annotation.pageIndex == displayedPage) {
13             // 转换页面坐标为实际屏幕坐标
14             RectF screenRect = new RectF(
15                 annotation.position.left * pageWidth,
16                 annotation.position.top * pageHeight,
17                 annotation.position.right * pageWidth,
18                 annotation.position.bottom * pageHeight
19             );
20             Paint paint = new Paint();
21             paint.setColor(Color.YELLOW);
22             paint.setStyle(Paint.Style.FILL);
23             paint.setAlpha(150);
24             canvas.drawRect(screenRect, paint);
25
26             // 绘制注释文字
27             paint.setColor(Color.BLACK);
28             paint.setTextSize(40);
29             canvas.drawText(annotation.content, screenRect.left,
screenRect.bottom, paint);
30         }
31     }
32 }

```

3. 添加注释 提供一个方法供外部调用，用于向页面添加注释：

```

1  java
2
3
4  复制代码
5  public void addAnnotation(int pageIndex, RectF position, String content) {
6      annotations.add(new Annotation(pageIndex, position, content));
7      invalidate(); // 重新绘制
8  }

```

3. 保存注释和高亮记录

你可以将高亮和注释信息保存到文件（如 JSON 或数据库）中，方便下次加载时恢复。

保存记录

将 `highlights` 和 `annotations` 转换为 JSON 并保存：

```
1 java
2
3
4 复制代码
5 Gson gson = new Gson();
6 String highlightJson = gson.toJson(highlights);
7 String annotationJson = gson.toJson(annotations);
8
9 // 保存到文件或数据库
```

加载记录

在重新打开 PDF 时加载这些记录：

```
1 java
2
3
4 复制代码
5 Type highlightType = new TypeToken<List<Highlight>>().getType();
6 Type annotationType = new TypeToken<List<Annotation>>().getType();
7
8 highlights = gson.fromJson(savedHighlightJson, highlightType);
9 annotations = gson.fromJson(savedAnnotationJson, annotationType);
10
11 invalidate(); // 重新绘制高亮和注释
```

4. 用户交互（添加高亮和注释）

1. 检测用户触摸位置 重写 `onTouchEvent` 方法，检测用户选择的区域：

```
1 java
2
3
4 复制代码
5 @Override
6 public boolean onTouchEvent(MotionEvent event) {
7     if (event.getAction() == MotionEvent.ACTION_UP) {
8         float x = event.getX();
9         float y = event.getY();
10        RectF selectedArea = calculateSelectedArea(x, y); // 计算选中的页面区域
11
12        // 触发高亮或注释逻辑
13        addHighlight(getCurrentPage(), selectedArea, Color.YELLOW);
14        addAnnotation(getCurrentPage(), selectedArea, "这是一个注释");
15    }
```

```
16     return super.onTouchEvent(event);  
17 }
```

2. **显示注释弹窗** 在用户点击注释区域时，显示一个弹窗：

```
1  java  
2  
3  
4  复制代码  
5  private void showAnnotationDialog(String content) {  
6      AlertDialog.Builder builder = new AlertDialog.Builder(getContext());  
7      builder.setTitle("注释");  
8      builder.setMessage(content);  
9      builder.setPositiveButton("关闭", null);  
10     builder.show();  
11 }
```

最终效果

- 高亮和注释可以通过用户交互动态添加。
- 注释内容在点击时会弹窗显示。
- 所有高亮和注释的数据可以持久化存储，方便下次加载时恢复。

如果需要更详细的交互逻辑或其他扩展功能，请随时告知！